Chapter 10

Programming Principles

APIs, JSON, and Security Principles

Mr. SOK Pongsametrey
metreysk@gmail.com

This week we connect API usage, JSON processing, and security principles to Programming Principles like separation of concerns, modularity, maintainability, and security. We'll explore how modern applications integrate with external services safely and efficiently.

INTEGRATING EXTERNAL SERVICES SECURELY AND EFFICIENTLY

What is an API?

- **API = Application Programming Interface**
- A contract that defines how software components should interact
- **Types of APIs:**
  - **REST APIs:** Representational State Transfer (most common)
  - **GraphQL APIs:** Query language for APIs
  - **SOAP APIs:** Simple Object Access Protocol (legacy)
  - **WebSocket APIs:** Real-time bidirectional communication
- **HTTP Methods:**
  - **GET** - Read data
  - **POST** - Create data
  - **PUT** - Update data
  - **DELETE** - Remove data
- **Benefits:** Code reuse, separation of concerns, scalability, specialization

Separation of Concerns, Modularity

RESTful API Fundamentals

- **REST Principles:**
  - **Stateless:** Each request contains all needed information
  - **Resource-based:** URLs represent resources, not actions
  - **HTTP methods:** Use appropriate verbs for operations
  - **Standard formats:** JSON for data exchange

URL Structure Examples:

```
GET /api/users          → Get all users
GET /api/users/123      → Get specific user
POST /api/users         → Create new user
PUT /api/users/123      → Update user
DELETE /api/users/123   → Delete user
```

HTTP Status Codes:

Code	Meaning
200	OK - Success
201	Created
404	Not Found
500	Server Error

Consistency Maintainability

Making HTTP Requests in Java

GET Request Example:

```
import java.net.http.*;
import java.net.URI;

HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(10))
        .build();

HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.example.com/users"))
        .header("Accept", "application/json")
        .timeout(Duration.ofSeconds(30))
        .GET()
        .build();

HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

if (response.statusCode() == 200) {
    System.out.println(response.body());
}
```

POST Request with JSON:

```
String jsonBody = "{\"name\":\"Alice\",\"age\":28}";

HttpRequest postRequest = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(jsonBody))
        .build();
```

Clear API Maintainability

What is JSON?

- **JSON = JavaScript Object Notation**
- Lightweight, text-based data interchange format
- **Human-readable** and **machine-parseable**
- **Data Types:**
  - String, Number, Boolean, null, Object, Array

Example Structure:

```
{
  "name": "Alice Johnson",
  "age": 28,
  "email": "alice@example.com",
  "isActive": true,
  "address": {
    "street": "123 Main St",
    "city": "New York"
  },
  "skills": ["Java", "Python", "JavaScript"],
  "projects": [
    {
      "name": "E-commerce Platform",
      "status": "completed"
    }
  ]
}
```

Readability Interoperability

JSON Processing in Java

Using org.json Library:

```
import org.json.*;

// Parse JSON string
JSONObject jsonObject = new JSONObject(jsonString);

// Extract simple values
String name = jsonObject.getString("name");
int age = jsonObject.getInt("age");

// Extract nested object
JSONObject address = jsonObject.getJSONObject("address");
String city = address.getString("city");

// Extract array
JSONArray skills = jsonObject.getJSONArray("skills");
for (int i = 0; i < skills.length(); i++) {
    System.out.println(skills.getString(i));
}

// Create JSON
JSONObject user = new JSONObject();
user.put("name", "Bob Smith");
user.put("age", 35);

JSONArray hobbies = new JSONArray();
hobbies.put("Programming");
user.put("hobbies", hobbies);
```

Structured Processing | Reliability

```
=== JSON Processing Demonstration ===

--- Example 1: Creating JSON Objects ---
[CREATED] Simple JSON object:
{
  "name": "Alice Johnson",
  "isActive": true,
  "age": 28,
  "email": "alice@example.com"
}

[CREATED] JSON with nested object:
{
  "address": {
    "zipCode": "10001",
    "city": "New York",
    "street": "123 Main St"
  },
  "name": "Alice Johnson",
  "isActive": true,
  "age": 28,
  "email": "alice@example.com"
}

--- Example 2: Parsing JSON Strings ---
[PARSED] JSON data:
Name: Bob Smith
Age: 35
Email: bob@example.com
Active: true
Salary: $75000.50

--- Example 3: Nested JSON Structures ---
[PARSED] Nested JSON data:
Name: Charlie Brown
Email: charlie@example.com
Phone: 555-1234
Location: Boston, MA
Coordinates: 42.3601, -71.0589
```

API Security Fundamentals

Security Threats

- **Unauthorized access:** Accessing APIs without permission
- **Data breaches:** Sensitive information exposure
- **Injection attacks:** SQL injection, NoSQL injection
- **Man-in-the-middle:** Intercepting communications
- **Denial of Service:** Overwhelming API with requests

Security Principles

- **Authentication:** Verify identity (who you are)
- **Authorization:** Verify permissions (what you can do)
- **Confidentiality:** Protect data in transit and at rest
- **Integrity:** Ensure data hasn't been tampered with
- **Availability:** Keep services accessible to legitimate users

Critical: Security is a cross-cutting concern that affects all layers of application design

Defense in Depth, Least Privilege

Authentication Methods

1. API Keys:

```
HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .header("X-API-Key", "your-api-key-here")
        .header("Accept", "application/json")
        .GET()
        .build();
```

2. Bearer Tokens (JWT):

```
HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .header("Authorization", "Bearer " + accessToken)
        .header("Accept", "application/json")
        .GET()
        .build();
```

3. Basic Authentication:

```
String auth = username + ":" + password;
String encodedAuth = Base64.getEncoder()
        .encodeToString(auth.getBytes());

HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .header("Authorization", "Basic " + encodedAuth)
        .GET()
        .build();
```

Standardized Authentication Security

OAuth 2.0 Deep Dive

OAuth 2.0 Flow

1. **Authorization Request:** Client redirects user to authorization server
2. **User Authorization:** User grants or denies permission
3. **Authorization Grant:** Server redirects back with code
4. **Access Token Request:** Client exchanges code for access token
5. **Access Token Response:** Server returns access token
6. **Protected Resource Access:** Client uses token to access API

Benefits of OAuth 2.0

- ✓ **No password sharing:** Apps never see user passwords
- ✓ **Limited scope:** Apps only get permissions they need
- ✓ **Token expiration:** Reduces risk of long-term compromise
- ✓ **Revocable access:** Users can revoke access anytime

Principle tie-in: OAuth promotes separation of concerns and least privilege principle (Separation of Concerns, Security)

HTTPS and Transport Security

Why HTTPS Matters

- **Encryption:** Protects data in transit using TLS/SSL
- **Authentication:** Verifies server identity
- **Integrity:** Prevents tampering with data
- **Trust:** Required for modern web applications

HTTPS Best Practices

- ✓ **Always use HTTPS** for production APIs
- ✓ **Validate certificates** properly
- ✓ **Use strong TLS versions** (TLS 1.2+)
- ✓ **Implement certificate pinning** for critical services
- ✗ **Never disable certificate validation** in production

Production Configuration:

```
HttpClient client = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(10))
        .build(); // Uses default SSL context with proper validation
```

Security by Design | Protection at All Layers

Secure API Design Patterns

1. Input Validation:

```
public void validate() throws ValidationException {
    if (email == null || !isValidEmail(email)) {
        throw new ValidationException("Invalid email");
    }

    if (name == null || name.trim().isEmpty()) {
        throw new ValidationException("Name required");
    }

    // Sanitize inputs
    this.name = sanitizeString(name);
}
```

2. Rate Limiting:

```
public class RateLimiter {
    private int maxRequests = 100;
    private long windowMs = 3600000; // 1 hour

    public boolean isAllowed(String clientId) {
        // Check if client exceeded rate limit
        return checkAndUpdateLimit(clientId);
    }
}
```

3. Error Handling:

```
// Return generic error to client (don't expose internals)
if (e instanceof ValidationException) {
    return new APIError("Invalid input", "VALIDATION_ERROR");
} else {
    return new APIError("Internal error", "INTERNAL_ERROR");
}
```

Defense in Depth Least Privilege

Real-world API Integration Example

Weather Service Integration

```
public class WeatherService {
    private final String apiKey;
    private final HttpClient httpClient;
    private final RateLimiter rateLimiter;

    public WeatherData getCurrentWeather(String city) {
        // Check rate limiting
        if (!rateLimiter.isAllowed("weather-service")) {
            throw new WeatherServiceException("Rate limit exceeded");
        }

        String url = String.format(
            "https://api.openweathermap.org/weather?q=%s&appid=%s",
            URLEncoder.encode(city), apiKey);

        HttpRequest request = HttpRequest.newBuilder()
                .uri(URI.create(url))
                .header("Accept", "application/json")
                .build();

        HttpResponse<String> response = httpClient.send(request,
                HttpResponse.BodyHandlers.ofString());

        return parseWeatherResponse(response.body());
    }
}
```

Modularity | Error Handling | Rate Limiting

The output will look something like this:

```
Fetching weather for London
Temperature: 15.2°C
Humidity: 72%
Description: Partly cloudy

Fetching weather for New York
Temperature: 22.8°C
Humidity: 65%
Description: Clear sky

Fetching weather for Tokyo
Temperature: 19.5°C
Humidity: 78%
Description: Light rain
```

API Testing and Monitoring

Unit Testing APIs:

```
@Test
public void testParseValidWeatherResponse() {
    String mockResponse = """
        {
            "name": "London",
            "main": {"temp": 15.5, "humidity": 65}
        }
        """;

    WeatherService service = new WeatherService("test-key");
    WeatherData data = service.parseWeatherResponse(mockResponse);

    assertEquals("London", data.city);
    assertEquals(15.5, data.temperature, 0.1);
}
```

API Monitoring:

```
public class APIMonitor {
    public <T> T monitorAPICall(String operation, Supplier<T> apiCall) {
        long startTime = System.currentTimeMillis();

        try {
            T result = apiCall.get();
            logSuccess(operation, System.currentTimeMillis() - startTime);
            return result;
        } catch (Exception e) {
            logError(operation, e);
            throw e;
        }
    }
}
```

Testing Reliability

- **Java Code: API Testing & Monitoring**
- **Unit Testing (testParseValidWeatherResponse)**
  - Tests **WeatherService** JSON parsing logic
  - Uses mock data to ensure correct API response handling
  - Validates reliability of data processing
- **API Monitoring (APIMonitor)**
  - Tracks API call status and performance
  - Records success/failure metrics
  - Ensures system reliability
- **Key Focus:** Testing & Reliability

API Design Best Practices

Resource-Based URLs:

✓ **Good:**
GET /api/users/123/posts
POST /api/posts
PUT /api/posts/456
DELETE /api/posts/456

✗ **Bad:**
GET /api/getUserPosts?userId=123
POST /api/createPost
POST /api/updatePost

Consistent Response Format:

```
public class APIResponseWrapper<T> {
    private boolean success;
    private T data;
    private String message;
    private List<String> errors;

    public static <T> APIResponseWrapper<T> success(T data) {
        return new APIResponseWrapper<>(true, data, null, null);
    }
}
```

Consistency | Developer Experience

Performance and Scaling

1. Connection Pooling:

```
HttpClient sharedClient = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(10))
        .executor(Executors.newFixedThreadPool(10))
        .build();
```

2. Asynchronous Processing:

```
public CompletableFuture<String> fetchDataAsync(String url) {
    return httpClient.sendAsync(request,
            HttpResponse.BodyHandlers.ofString())
            .thenApply(HttpResponse::body);
}
```

3. Caching:

```
public class APICache {
    private ConcurrentHashMap<String, CachedResponse> cache;
    private long defaultTTL = 300_000; // 5 minutes

    public String get(String key) {
        CachedResponse cached = cache.get(key);
        return (cached != null && !cached.isExpired())
            ? cached.getData() : null;
    }
}
```

Efficiency Scalability

Connection Pooling:

- Reuses HTTP connections across requests
- Eliminates TCP handshake overhead per call
- Prevents socket exhaustion under load
  Impact: Reduces latency by 30-50% in high-throughput scenarios.

Asynchronous Processing:

- Non-blocking I/O operations
- Parallelizes API calls
- Avoids thread-blocking during network waits
  Impact: Scales 3-5x more requests per second vs. synchronous calls.

Caching:

- Eliminates redundant API calls
- Reduces network latency and API costs
- Thread-safe cache population
  Impact: Cuts API load by 60-80% for repeated requests.

Key Performance Metrics

TECHNIQUE	THROUGHPUT GAIN	LATENCY REDUCTION	RESOURCE EFFICIENCY
Connection Pooling	2-3x	40-60%	↓ Socket exhaustion
Async Processing	3-5x	50-70%	↑ CPU utilization
Caching	N/A	70-90%	↓ Network I/O

TECHNIQUE	DESCRIPTION	JAVA IMPLEMENTATION EXAMPLE	WHEN TO USE	THROUGHPUT GAIN	LATENCY REDUCTION	RESOURCE EFFICIENCY
Connection Pooling	Reuses HTTP connections to avoid TCP handshake overhead per request	`HttpClient.newBuilder().connectTimeout(Duration.ofSeconds(5)).build()` (Java 11+)	Frequent HTTP requests to same endpoints	2-3x	40-60%	Prevents socket exhaustion
Async Processing	Non-blocking operations allowing threads to handle multiple requests concurrently	`CompletableFuture.supplyAsync(() -> fetchData(url))` + `.thenApply()` chaining	I/O-bound workloads with waiting periods	3-5x	50-70%	Improves CPU utilization (reduces thread count)
Caching	Stores responses in memory to avoid redundant API calls	`ConcurrentHashMap<String, String> cache` with computeIfAbsent()	Frequent repeated requests with stable data	N/A	70-90%	Reduces network I/O and API rate limiting

Key Insights:

1. **Connection Pooling** is most effective for HTTP-heavy applications
2. **Async Processing** shines in high-latency environments (e.g., microservices)
3. **Caching** provides the highest latency reduction but requires careful cache invalidation
4. **Combined Effect**: Using all three techniques together can achieve 5-10x throughput improvement while reducing latency by 80-95%

Throughput = Number of Successful Operations (e.g., API Calls) Completed per Second

(Units: Requests Per Second / RPS, Operations Per Second / OPS)

Circuit Breaker Pattern

Protect Your System from Cascading Failures

Circuit States

- **CLOSED:** Normal operation, requests pass through
- **OPEN:** Too many failures, requests blocked
- **HALF_OPEN:** Testing if service recovered

Implementation

```
public class CircuitBreaker {
    private enum State { CLOSED, OPEN, HALF_OPEN }

    private volatile State state = State.CLOSED;
    private final int failureThreshold;
    private final long timeoutMs;
    private AtomicInteger failureCount = new AtomicInteger(0);

    public <T> T execute(Supplier<T> operation) throws Exception {
        if (state == State.OPEN) {
            if (shouldAttemptReset()) {
                state = State.HALF_OPEN;
            } else {
                throw new RuntimeException("Circuit breaker is OPEN");
            }
        }

        try {
            T result = operation.get();
            onSuccess();
            return result;
        } catch (Exception e) {
            onFailure();
            throw e;
        }
    }
}
```

Resilience | Fault Tolerance

Future Trends in API Development

Emerging Technologies

1. GraphQL Integration:

- Query exactly the data you need
- Reduce over-fetching and under-fetching
- Strong typing and introspection

2. Serverless API Integration:

- Function-as-a-Service (AWS Lambda, Azure Functions)
- Event-driven architecture with API gateways
- Auto-scaling based on demand
- Pay-per-use pricing model

3. AI-Powered APIs:

- Natural Language Processing services
- Computer Vision APIs
- Machine Learning model serving
- Automated API testing and optimization

4. API Security Evolution:

- Zero Trust Architecture principles
- API Gateway security patterns
- Machine Learning for threat detection
- Blockchain for API authentication

Lab Activity - News Aggregator Service

Build a secure news aggregation service

Requirements:

- **Multiple News Sources:** Integrate with 2-3 news APIs
- **Authentication:** Implement secure API key management
- **Rate Limiting:** Respect API rate limits
- **Error Handling:** Graceful handling of failures
- **JSON Processing:** Parse and normalize data
- **Caching:** Implement simple caching
- **Security:** Input validation and secure HTTP

Core Classes

```
public interface NewsSource {
    List<Article> getTopHeadlines(String category);
    List<Article> searchArticles(String query);
}

public class NewsAggregator {
    private List<NewsSource> sources;
    private ArticleCache cache;

    public List<Article> getAggregatedNews(String category) {
        // Aggregate from multiple sources
    }
}
```

Security | Modularity | Reliability

Exercise: Social Media Integration Platform

Build a multi-platform social media integration

Core Requirements

1. **Multi-Platform Support:** Twitter, Reddit, GitHub APIs
2. **OAuth 2.0:** Implement proper authentication
3. **Unified Data Model:** Common interfaces for different platforms
4. **Security:** Secure credential storage, input validation
5. **Advanced Features:** Async processing, caching, circuit breaker

Implementation Structure

```
public interface SocialMediaClient {
    List<SocialMediaPost> getRecentPosts(int limit);
    SocialMediaPost getPost(String id);
    boolean isHealthy();
}

public class SocialMediaAggregator {
    private List<SocialMediaClient> clients;

    public CompletableFuture<List<SocialMediaPost>>
        getAggregatedFeed(int postsPerSource) {
        // Parallel API calls with circuit breaker
    }
}
```

Wrap-up and Key Takeaways

Key Takeaways

- **APIs are contracts** that enable system integration and modularity
- **JSON is the standard** for modern data exchange
- **Security must be built-in** from the start, not added later
- **OAuth 2.0 provides secure** delegated authentication
- **HTTPS is mandatory** for production API communications
- **Testing and monitoring** are essential for reliable integrations

Programming Principles Applied:

Separation of Concerns: APIs separate client and server responsibilities

Modularity: Services can be developed and deployed independently

Reusability: Well-designed APIs can serve multiple clients

Security: Defense-in-depth with multiple security layers

Security Checklist:

✓ **Use HTTPS** for all API communications
✓ **Implement proper authentication** (OAuth 2.0, API keys)
✓ **Validate all inputs** and sanitize data
✓ **Use rate limiting** to prevent abuse
✓ **Keep dependencies updated** for security patches